



Wrocław
University
of Science
and Technology

Software Engineering

Introductory lecture

Marcin Jodłowiec

October 8th, 2025

Agenda

- Subject objectives
- Lecture outline: Topics covered
- Course completion rules
- Literature
- Basic terms and definitions
- Software lifecycle phases
- Software process models

Subject objectives I

Subject objectives

- C1 Familiarity with basic concepts of software engineering
- C2 Familiarity with requirements engineering principles and techniques
- C3 Familiarity with main aspects of modelling, design and testing of information systems

Subject objectives II

Subject educational effects

Knowledge-related:

K1INF_W14 Has detailed knowledge of software and database design

Skills-related:

K1INF_U03 Can describe **requirements** and design – using the selected **modeling language** – a general **software architecture** and a **database schema**

K1INF_U04 Is able to implement, in accordance with the design, software and database for simple, typical applications and verify the correctness of the solution.

K1INF_U21 He can cooperate with other people as part of a team undertaking.

Lecture outline: Topics covered I

- ▶ **Lecture 1** [Introduction to Software Engineering](#). Course overview. Basic terms and defs. Software lifecycle phases. Overview of development methodologies.
- ▶ **Lecture 2** [Project management and software development planning](#). Team roles and organization. Scheduling and estimation (time, cost, and resources). Estimation techniques and cost modeling. Risk analysis and mitigation.
- ▶ **Lecture 3** [Requirements Engineering: traditional approach](#). User needs analysis. Types of requirements (functional and non-functional). Elicitation techniques: interviews, questionnaires, observation. Requirement classification and prioritization (FURPS, MoSCoW, Kano model).
- ▶ **Lecture 4** [Requirements Engineering: agile approach](#). Product backlog, epics, and user stories. Acceptance criteria and prioritization. Managing change in agile projects. Stakeholder collaboration and feedback cycles.

Lecture outline: Topics covered II

- ▶ **Lecture 5** [Software architecture](#). Architectural styles: layered (n-tier), event-driven, service-oriented, microservices. Architectural decisions and trade-offs. Modeling software architecture with the C4 model and Architecture-as-Code (AaC). Model-based engineering.
- ▶ **Lecture 6** [UML-based modeling of structure](#). Class diagram, object diagram, component diagram, package diagram and deployment diagram. Principles of mapping domain concepts to software structure. Attributes, associations, compositions, and dependencies.
- ▶ **Lecture 7** [UML-based modeling of behavior](#). Sequence diagrams, activity diagrams, and state diagrams. Scenarios of interaction between system components. Control and data flow modeling. Relationship between structural and behavioral models.
- ▶ **Lecture 8** [Database design](#). Logical and physical modeling. Relational vs. non-relational databases (SQL, NoSQL). Database integration with application layers.

Lecture outline: Topics covered III

- ▶ **Lecture 9** [API and contract design](#). REST, GraphQL, and asynchronous API communication. OpenAPI and AsyncAPI specifications. Contract testing, versioning, and practices in service design.
- ▶ **Lecture 10** [Classical OOP design patterns](#). Categories: creational, structural, behavioral. Selected patterns: Singleton, Factory, Adapter, Observer, Strategy. Role of design patterns in improving code quality and maintainability.
- ▶ **Lecture 11** [Modern architectural patterns and design approaches](#). Clean Code principles, Domain-Driven Design concepts, Hexagonal Architecture, CQRS, Event Sourcing, and some patterns such as Dependency Injection and Repository.
- ▶ **Lecture 12** [DevOps and CI/CD principles](#). Automation of building, testing, and deployment. Continuous Integration, Continuous Delivery, Continuous Deployment. The 12-Factor App methodology. Configuration management and Infrastructure as Code.

Lecture outline: Topics covered IV

- ▶ **Lecture 13** [Software documentation and measurement](#). Types of documentation (technical, architectural, user-oriented). Software metrics and quality indicators. Tools for static analysis and code quality assessment.
- ▶ **Lecture 14** [Quality management and software testing](#). Testing levels: unit, integration, system, acceptance. Test-Driven Development (TDD) and Behavior-Driven Development (BDD). Automated testing, coverage analysis, and reporting.
- ▶ **Lecture 15** [Current trends in software engineering](#). Serverless and edge computing. Role of AI in software development: code generation, code review, and test automation. Emerging paradigms and future directions in software engineering.

Course completion rules

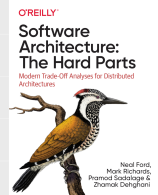
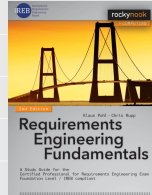
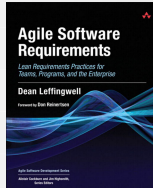
Project

- ▶ Teamwork: project assignment in teams (up to 6 people).
- ▶ Four phases:
 - ▶ elaboration of application concepts
 - ▶ requirements specification
 - ▶ design
 - ▶ implementation & testing
- ▶ Detailed grading rules are determined by the project instructor.

Lecture & Examination

- ▶ Slides will be published in MS Teams
- ▶ Exam: multiple choice test (regular and retake), dates will be scheduled asap.
- ▶ Grading according:

Score range	Grade
<0%, 50%)	2.0
<50%, 60%)	3.0
<60%, 70%)	3.5
<70%, 80%)	4.0
<80%, 90%)	4.5
<90%, 100%)	5.0



Other references:

- ▶ Software Engineering Body of Knowledge (SWEBOK)
- ▶ ISO/IEC/IEEE 12207:2017 - Software life cycle processes
- ▶ Unified Modeling Language (UML) Specification
- ▶ Slides from the lecture.

Software engineering – definitions I

Software engineering

Fritz Bauer (1969) Software Engineering is the establishment and use of sound engineering principles in order to obtain economically software that is reliable and works efficiently on real machines.

Dennis (1975) Software Engineering is the application of principles, skills and art to the design and construction of programs and system of programs.

IEEE (1991) Software Engineering is the application of a systematic, disciplined and quantifiable approach to the development, operation and maintenance of software.

Stephen Schach (1992) Software Engineering is a discipline whose aim is the production of quality software, that is delivered on time, within budget and that satisfies its requirements.

Ian Sommerville (2016) Software engineering is an **engineering discipline** with **all aspects of software production** from the early stages of system specification through to maintaining the system after it has gone into use.

David Farey (2021) Software engineering is the application of an empirical, scientific approach to finding efficient, economic solutions to practical problems in software.

Software engineering – definitions II

Key concepts

- ▶ **Engineering discipline:** Engineers apply theories, methods, and tools to make systems work – balancing quality, time, and cost.
- ▶ **All aspects of production:** Not only development, but also project management, methods, and tools supporting software development.
- ▶ Software engineers aim to produce high-quality results within schedule and budget.

Types of software products I

Two main categories

Generic products Stand-alone systems developed by a software organization and sold on the open market to any customer. Examples: mobile apps, office software, databases, project management tools, or “vertical” market applications (e.g. library, accounting, or dental record systems).

Customized software Systems commissioned and developed for a specific customer or organization. Examples: control systems for devices, enterprise process automation, air traffic control systems.

Difference: Generic products are **built for a market**; customized systems are built for a **specific client's needs**.

Types of software products II

Types of application systems (Ian Sommerville, 2016)

1. **Stand-alone applications** – Run locally on a PC or mobile device; self-contained functionality. *Examples:* office suites, CAD, photo editors, productivity apps.
2. **Interactive transaction-based applications** – Remote systems accessed via browser or app; often include data storage. *Examples:* e-commerce, business systems, cloud services.
3. **Embedded control systems** – Software controlling and managing hardware devices. *Examples:* automotive ABS, phone firmware, microwave controller.
4. **Batch processing systems** – Handle large volumes of data in batches. *Examples:* billing, payroll.
5. **Entertainment systems** – Focus on user interaction and experience. *Examples:* computer and console games.
6. **Modeling and simulation systems** – Used for scientific and engineering analysis of physical processes. *Examples:* weather models, fluid dynamics, traffic simulation.
7. **Data collection and analysis systems** – Gather sensor data and analyze it locally or in the cloud. *Examples:* IoT monitoring, industrial analytics, big data platforms.
8. **Systems of systems** – Large-scale integrations of independent subsystems. *Examples:* ERP + CRM + logistics management in enterprises.

Software engineering for different system types

Different systems – different needs

- ▶ **Embedded systems** – safety-critical, hard to update → need strong testing and validation.
- ▶ **Web / mobile apps** – fast-changing, user-focused → use iterative and component-based development.
- ▶ **Systems of systems** – complex integrations → require detailed planning and defined interfaces.

Common fundamentals

- ▶ **Planned process** – know what, how, and when to deliver.
- ▶ **Dependability** – reliable, secure, efficient operation.
- ▶ **Requirements control** – clear understanding of what users need.
- ▶ **Reuse** – use existing components and tools when possible.

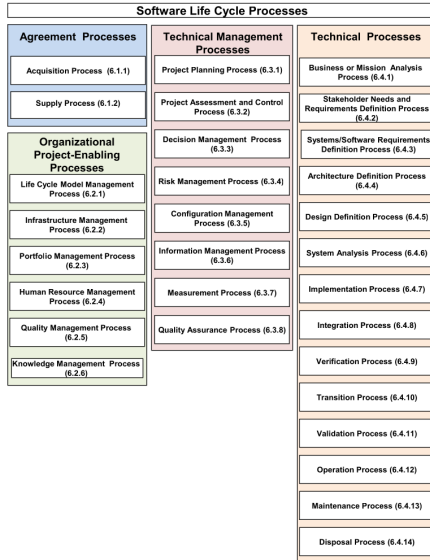
Software process

Software process (or software development process) is a set of related activities that lead to the production of software product.

Fundamental SE activities

1. **Specification** – defining system requirements and constraints
→ must be **defined**.
2. **Development** – software design and implementation
→ must **meet the specification**.
3. **Validation** – checking that software meets customer requirements
→ must **be validated**.
4. **Evolution** – modifying software to reflect new needs
→ must **adapt to change**.

Software processes according to ISO/IEC 12207



Core activities: Software Specification

Requirements engineering

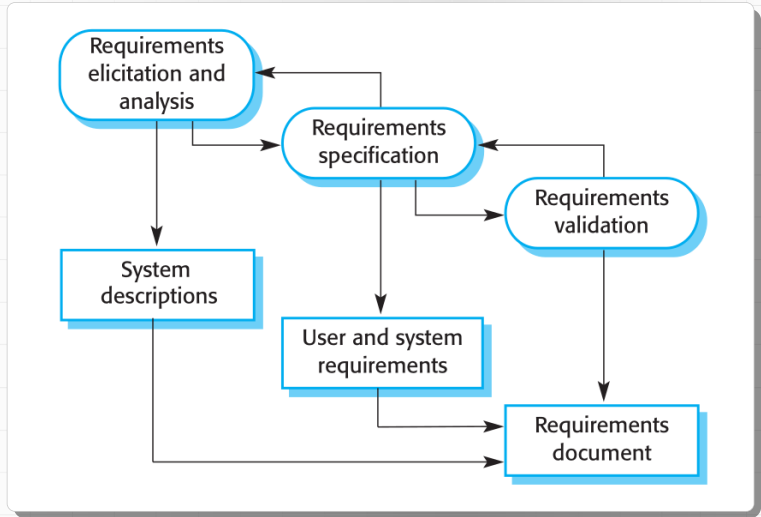


Figure: Requirements engineering process (Ian Sommerville, Software Engineering)

Core activities

Development

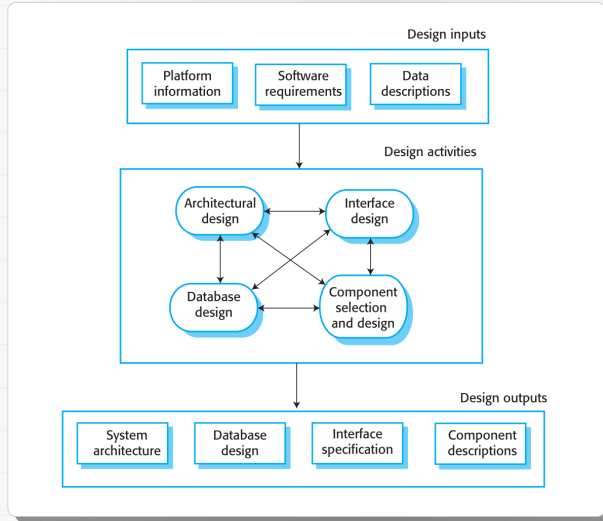


Figure: Design process (Ian Sommerville, Software Engineering)

Core activities

Validation

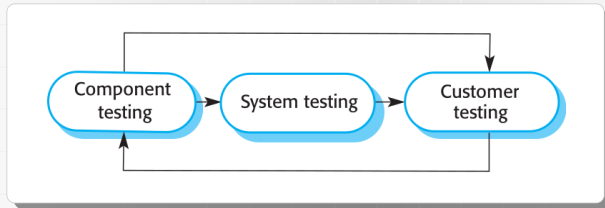


Figure: Testing process (Ian Sommerville, Software Engineering)

Core activities

Evolution

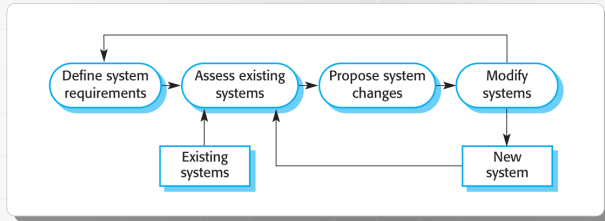


Figure: Evolution process (Ian Sommerville, Software Engineering)

Software process models

Software process model (or Software Development Life Cycle – SDLC model) is an **abstraction** of a process.

Software process models

Waterfall model Separate and distinct phases of specification and development. Progress is seen as flowing steadily downwards (like a waterfall) through these phases.

Incremental development Specification, development and validation are interleaved. The system is developed as a series of versions (increments) with each version adding functionality to the previous version.

Integration and configuration The system is assembled from existing components and systems, with as little new development as possible.

Spiral model Process is represented as a spiral rather than as a sequence of activities with backtracking. Each loop in the spiral represents a phase of the software process. The main distinguishing feature of the spiral model is its explicit recognition of risk and the need to assess risk and take appropriate action.

Software Process Models

Waterfall model

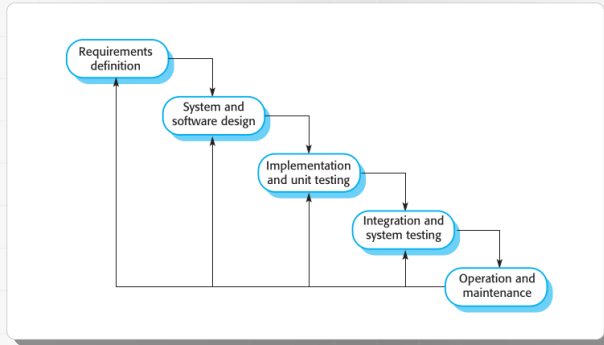


Figure: Waterfall model (Ian Sommerville, Software Engineering)

Software Process Models

Incremental development

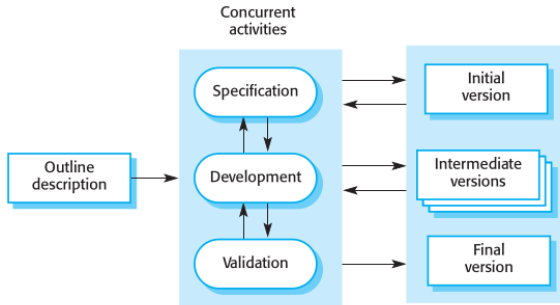


Figure: Incremental development (Ian Sommerville, Software Engineering)

Software Process Models

Integration and configuration

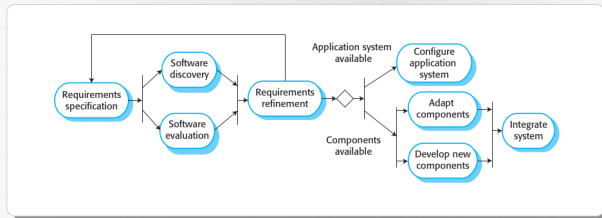


Figure: Integration and configuration (Ian Sommerville, Software Engineering)

Software Process Models

Spiral model

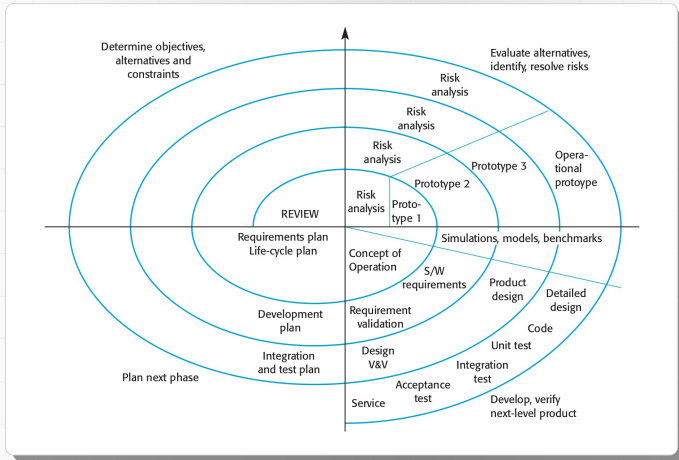


Figure: Spiral model (Ian Sommerville, Software Engineering)

It's all for today!

